RICHFIELD
richfield.ac.za

Name and Surname: Linda Mhlangu

Student ITS No: 402412302

Qualification: _____ **Year of Study:** _____ **Semester:** _____

Assignment due date: _____ **Date submitted:** _____

QUESTION	EXAMINER MARKS	MODERATOR MARKS	REMARKS

ASSIGNMENT INSTRUCTIONS

Please tick each box to confirm completion.

Use Times New Roman font, size 12, with 1.5 line spacing throughout the document.
Apply Harvard Referencing Style for all citations and references.

For essay-style assignments, please include the following sections:

Table of Contents
Introduction
Main Body (with relevant subheadings)
Conclusion
References

Submit the assignment in PDF format on Moodle.
Use the specified cover page provided.
Include a signed declaration of originality.

DECLARATION OF ORIGINALITY:
I hereby declare that this assignment is my own work and has not been copied from any other source except where due acknowledgment is made. I affirm that all sources used have been properly cited and that this submission complies with the institution's policies on academic integrity and plagiarism.

Student Signature: _____ **Date:** _____

Richfield Graduate Institute of Technology (Pty) Ltd is registered with the Department of Higher Education & Training as a Private Higher Education Institution under the Higher Education Act, 1997, Registration Certificate No. 2000/HE07/008.

Smart Pantry Manager

Mobile App Development 700 implementation report

Linda Mhlangu

Student number 402412302

Prepared 25 October 2026

This report explains the Java Android application, the database and matching rules, and the evidence collected from the running system. The app meets the required five-screen scope using the patterns covered in the supplied module material.

Repository: https://github.com/LinMhlaDoRh/smart-pantry-manager. The repository is private. Marker access must be granted before submission.

Preparation note: Codex assisted with implementation, testing and preparation of this report. The student must review and understand the code, acknowledge assistance according to institutional requirements, complete the cover declaration and record the narrated demonstration. The declaration has not been signed on the student's behalf.

Table of contents

1 Introduction 4

2 System design 5

3 Screenshots and functional evidence 7

4 Key Java code 14

5 Challenges and solutions 16

6 Conclusion and reflection 16

7 References 17

1 Introduction

Smart Pantry Manager helps a person track ingredients already available at home and identify meals that can be prepared from those ingredients. The intended users are households and students who want to make practical use of leftover food. The application implements the defined Smart Pantry Manager brief rather than introducing a different app concept (Richfield Graduate Institute of Technology, n.d.a).

The central rule is strict matching. A recipe appears only when all of its required ingredients are in the pantry in sufficient quantity. A recipe with even one missing ingredient is excluded. The app includes 20 recipes stored in SQLite, each with a name, measured ingredients and preparation steps.

The implementation uses Java Activities and XML layouts. Pantry data is held in SQLite through SQLiteOpenHelper, while a small SharedPreferences file stores the pantry sort order. These choices follow the module examples for Activities, Intents, persistent data and list adapters (Richfield Graduate Institute of Technology, n.d.b).

The five screens are Pantry List, Add/Edit Ingredient, Suggested Recipes, Recipe Detail and Settings. The app supports create, read, update and delete operations. It provides form errors, empty-list feedback and confirmation before deletion. Expiry dates are optional in the brief and are not included in this implementation.

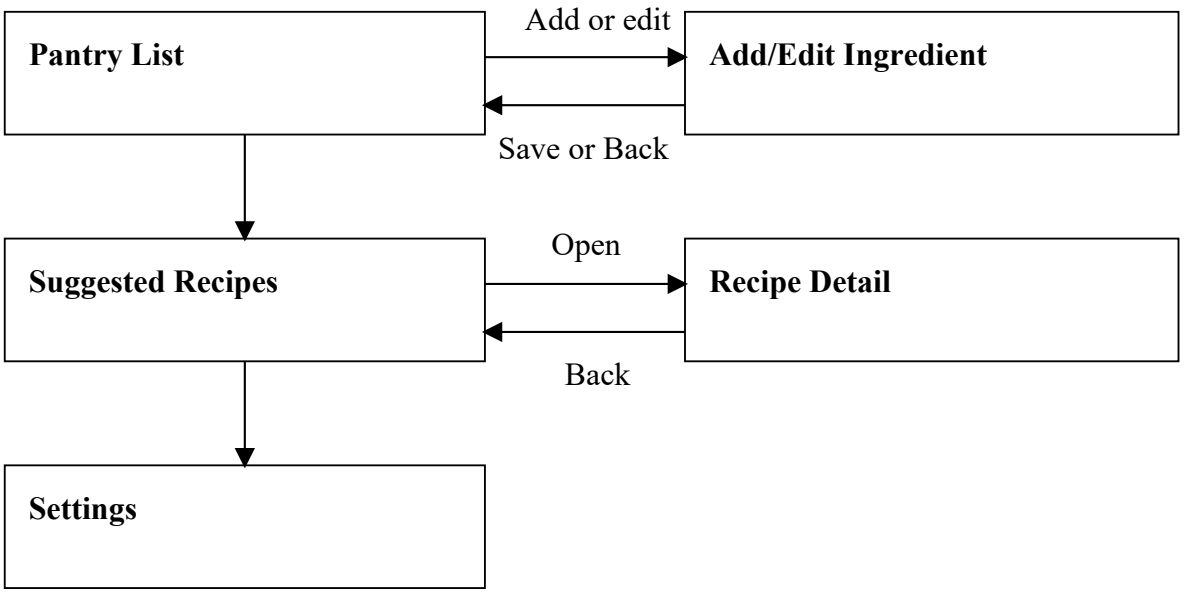
No account, external recipe service, cloud backend, mapping feature, location service, payment processing or Play Store publication is needed for this scope.

Database choice

SQLite was selected because the records belong to one device, the recipe collection is small and the app must work without a network connection. SQLiteOpenHelper is already covered in the module. ContentValues and Cursor provide direct, understandable database operations without a server or an additional persistence framework (Google, n.d.a).

2 System design

Screen flow



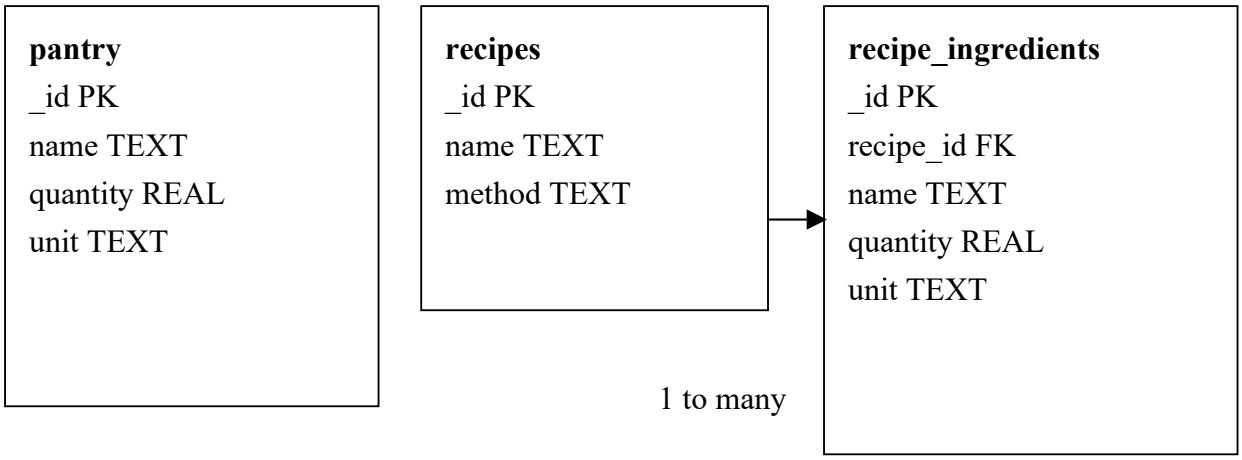
Pantry, Recipes and Settings share a bottom navigation bar. The arrows between these main screens represent navigation; the shared bar also allows direct movement between any of the three. Add/Edit Ingredient and Recipe Detail return to the previous screen with Back. IDs travel through explicit Intent extras rather than copying an entire record between Activities.

Layout and state

XML LinearLayouts arrange the widgets. ScrollViews keep the form, recipe method and settings usable on smaller screens. RecyclerViews display pantry rows and matching recipes through custom adapters. Shared string, colour and style resources keep the screens consistent.

onCreate loads each layout, links widgets and attaches listeners. PantryActivity and RecipesActivity refresh their database-dependent displays in onResume. This applies edits and settings when returning to a screen. IngredientActivity fills existing values only on first creation, allowing Android to restore unsaved widget state after rotation. Shared padding accounts for system bars and the keyboard.

Data model



Each pantry row has its own ID, ingredient name, positive quantity and unit. Each recipe has one or more requirement rows. recipe_ingredients.recipe_id references recipes._id, so one recipe can require several ingredients. SQLite foreign-key enforcement is enabled. A recipe deletion would remove its requirement rows through the declared cascade, although recipe editing is outside the app interface.

Pantry rows are independent of the recipe table. The matching method compares their normalised names and compatible measurement groups. Separate pantry entries can contribute to the available total. Recipe data is seeded with Java arrays and SQLite inserts during first database creation, and is not duplicated when the app reopens.

Strict matching decisions

Names are trimmed, repeated spaces are collapsed and comparison ignores case. A small explicit alias list recognises common plurals, including tomato/tomatoes and banana/bananas. This is limited, readable normalisation rather than a general language-processing system.

Mass is compared in grams and volume in millilitres. kg converts to g and l converts to ml. Counted items use pcs. The app does not assume that a given mass equals a given volume or number of pieces. A recipe is rejected when any available quantity is below the total required quantity. Matching never subtracts food or claims that a recipe has already been cooked.

3 Screenshots and functional evidence

The following images were captured from the actual app running on an Android 15 API 35 emulator. They show real records and interactions rather than mockups.

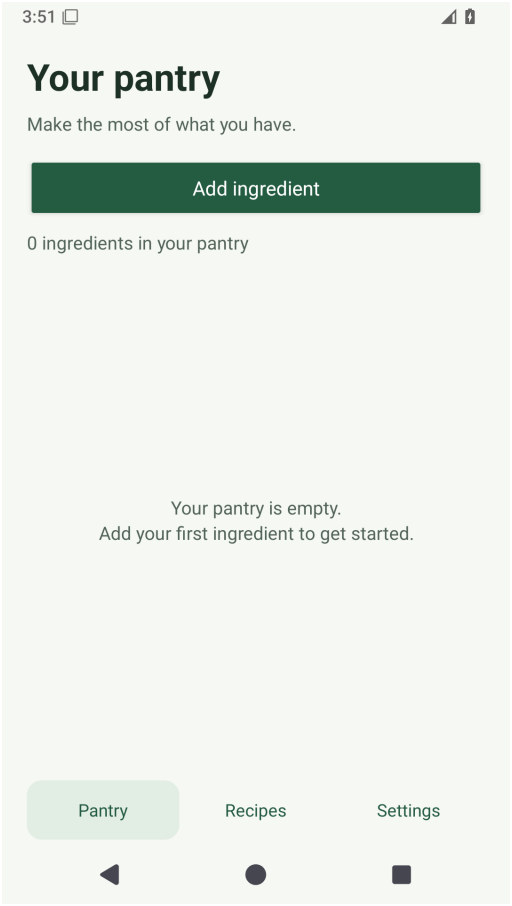


Figure 1. The first launch has an empty pantry and an Add ingredient action.

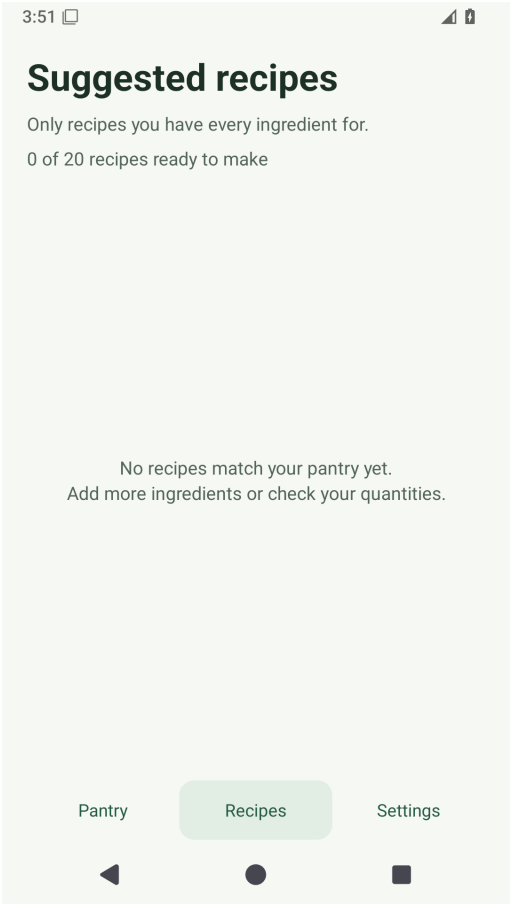


Figure 2. No recipes are suggested when the pantry has no ingredients.

Input validation

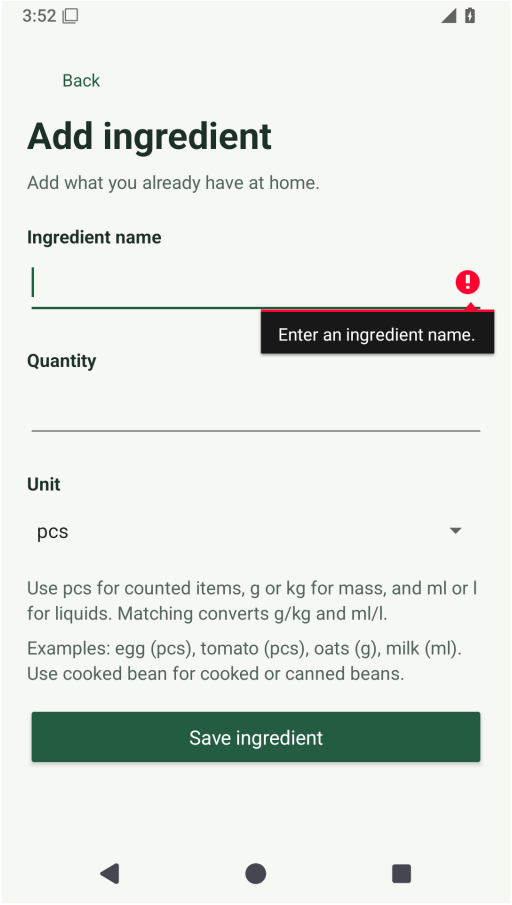


Figure 3. Saving a blank name produces a field error.

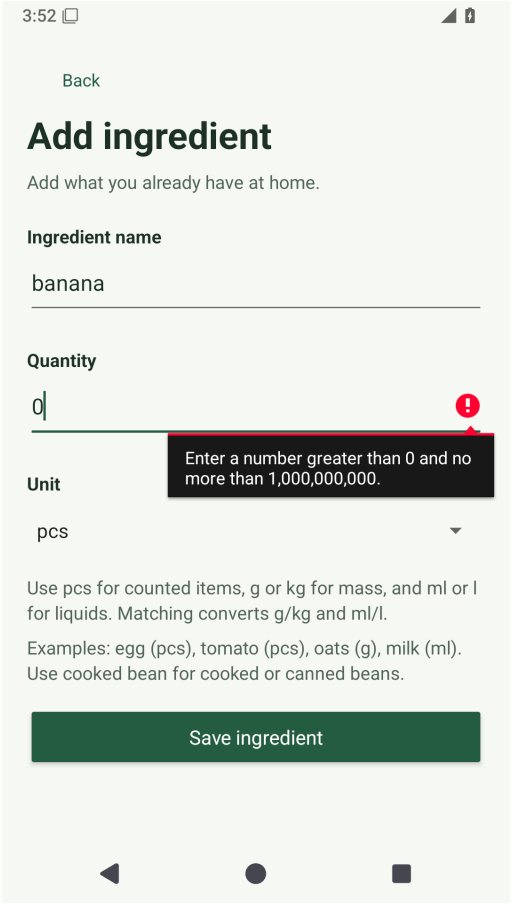


Figure 4. A zero quantity is rejected before any database insert.

Create and read

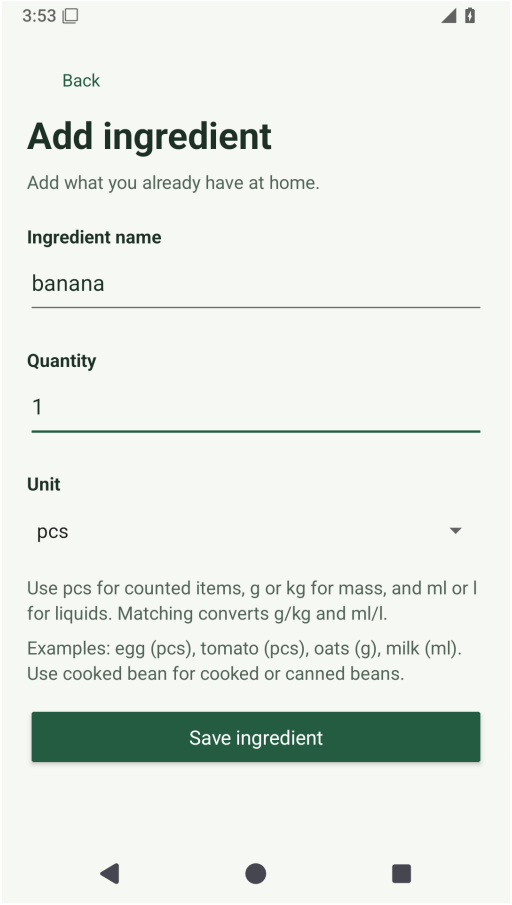


Figure 5. The form captures banana, quantity 1 and unit pcs.

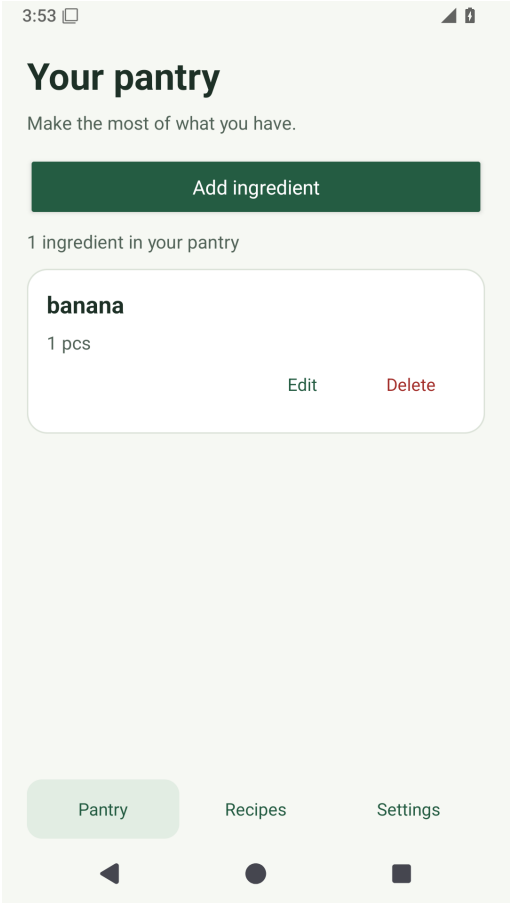


Figure 6. The saved database record appears in the custom pantry list.

Strict matching and update



Figure 7. Banana plus 0.2 l milk is insufficient for the recipe requiring 250 ml.

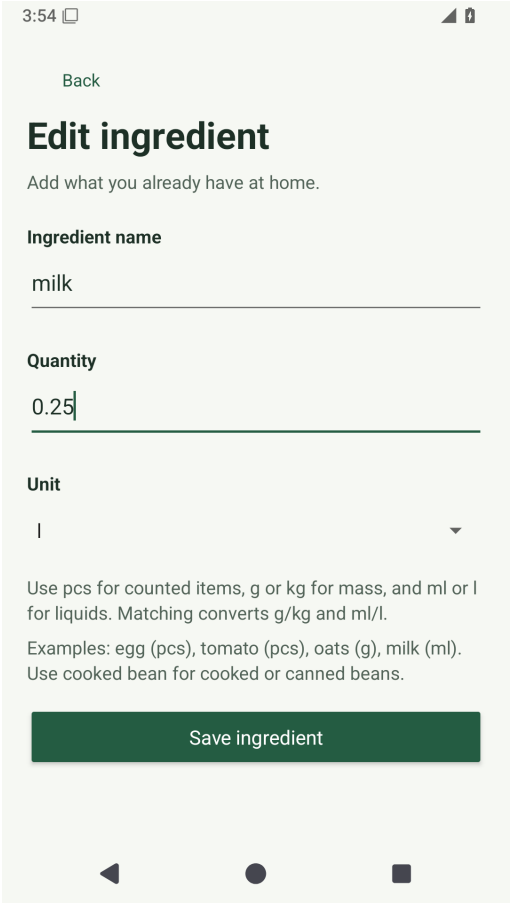


Figure 8. Editing milk to 0.25 l changes the existing record rather than adding another.

Suggestions and recipe detail

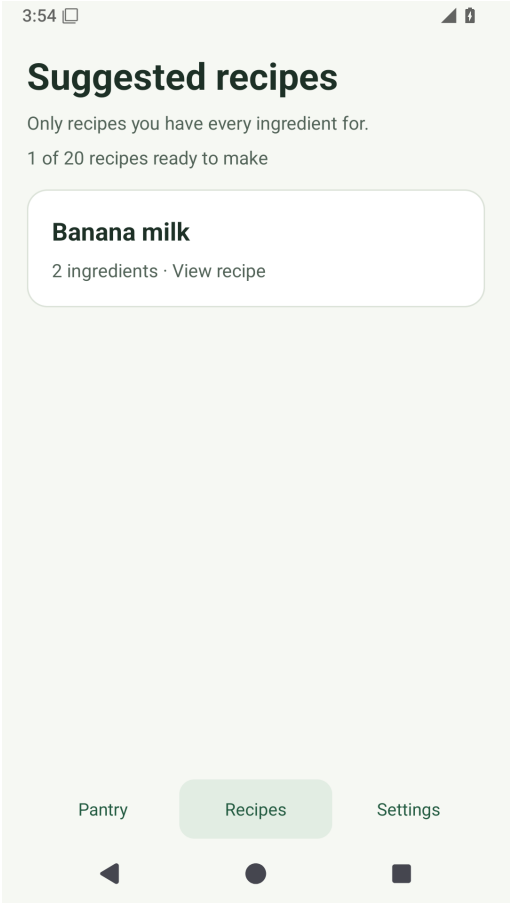


Figure 9. Banana milk appears after every ingredient is present in sufficient quantity.

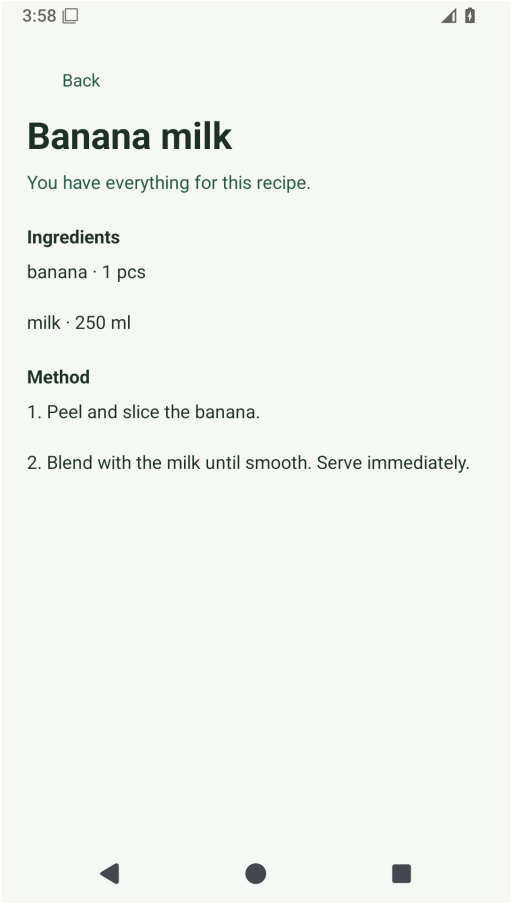


Figure 10. The detail screen shows the complete ingredient requirements and method.

Settings and persistence

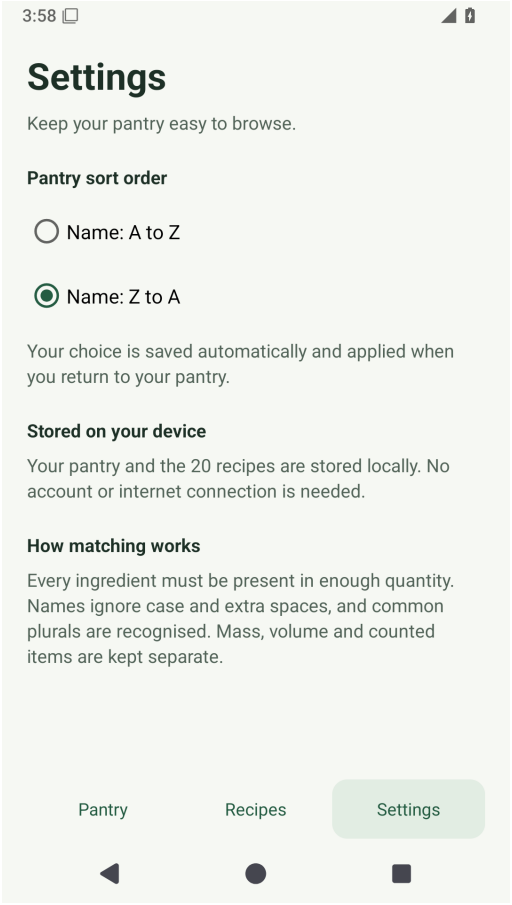


Figure 11. Settings saves the Z to A sort choice through SharedPreferences.

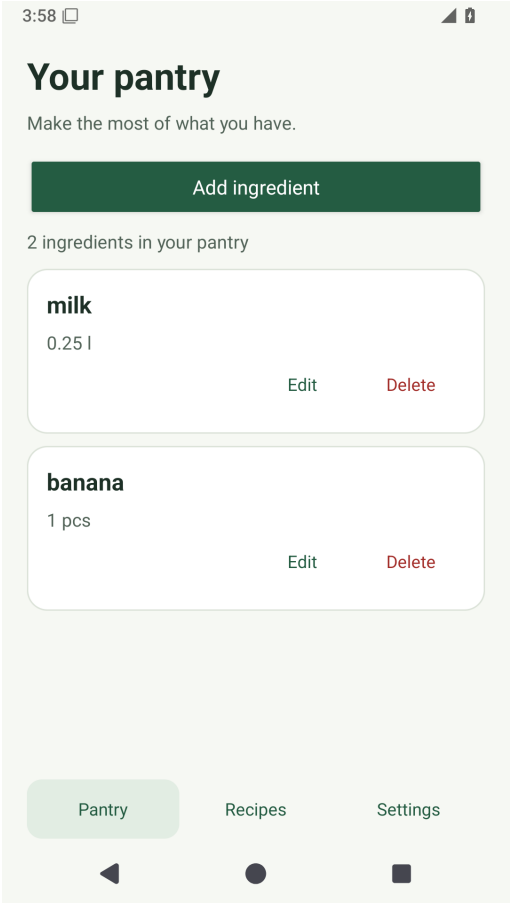


Figure 12. The two pantry records and their sort order remain after force stopping and reopening.

Delete and refreshed suggestions

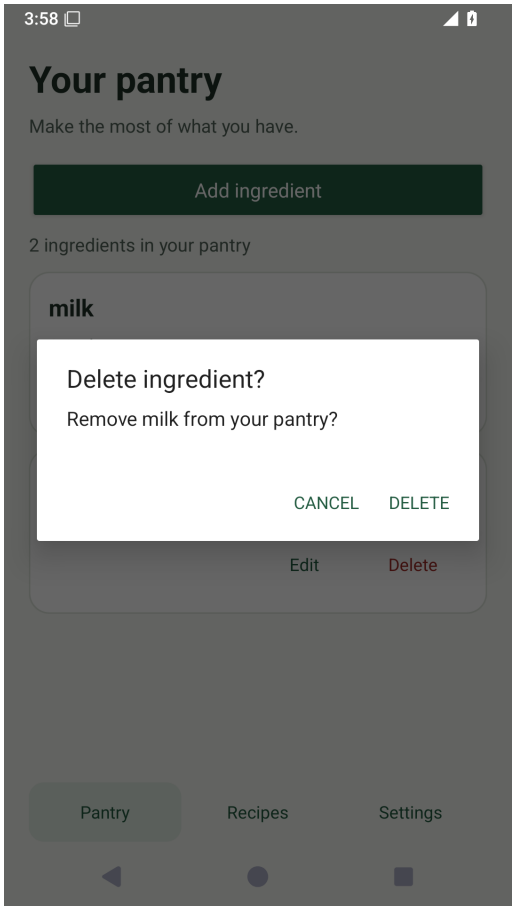


Figure 13. The app asks before deleting milk. Cancel was also checked and preserved the record.

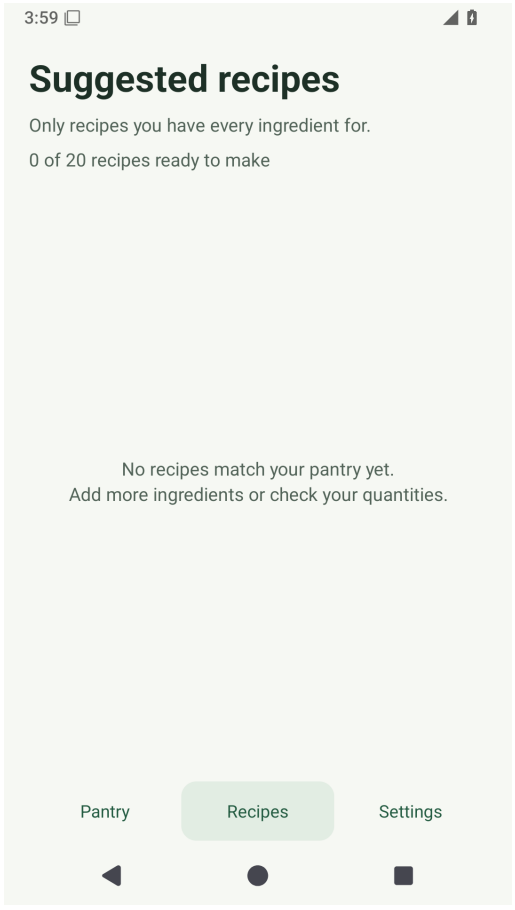


Figure 14. Once milk is deleted, the previously matching recipe is removed from suggestions.

4 Key Java code

1 SQLite create operation

```
public long addIngredient(Ingredient ingredient) {
   return getWritableDatabase().insertOrThrow(
      "pantry", null, valuesFor(ingredient));
}
```

PantryDatabase validates the model and creates ContentValues with the name, quantity and unit. insertOrThrow returns the inserted row ID. The UI displays success only after the operation succeeds. Update and delete select a particular row with its ID and a bound parameter, so an operation does not affect every pantry row (Google, n.d.a).

2 Strict matching rejection

```
if (available < needed) {
   return false;
}
```

This condition is inside RecipeMatcher.matches. For each required ingredient, the method totals compatible pantry quantities and the corresponding recipe requirement. A shortage returns false immediately. The method reaches true only after every requirement passes. The suggestions screen adds only recipes for which this complete check returns true.

3 Intent data exchange

```
Intent intent = new Intent(this, IngredientActivity.class);
intent.putExtra(IngredientActivity.EXTRA_INGREDIENT_ID,
   ingredient.getId());
startActivity(intent);
```

PantryActivity uses an explicit Intent to open the edit screen. Passing the ID identifies the stored row. IngredientActivity reads the long extra and queries SQLite to fill its form. The same pattern passes a recipe ID to RecipeDetailActivity.

4 Adapter binding

```
Ingredient ingredient = ingredients.get(position);
holder.name.setText(ingredient.getName());
holder.quantity.setText(
   BaseActivity.quantityText(ingredient));
```

IngredientAdapter binds the current data item to the widgets cached in its ViewHolder. onCreateViewHolder inflates row_ingredient.xml, onBindViewHolder fills that row, and getItemCount returns the list size. Edit and delete callbacks operate on the bound ingredient object (Google, n.d.b).

5 Saving the settings preference

```
preferences.edit()
   .putBoolean("sort_descending",
      checkedId == R.id.sortDescending)
   .apply();
```

SettingsActivity writes the selected ordering using a stable key and Boolean value. PantryActivity reads the same key in onResume and requests the corresponding SQLite ordering. This is why returning with Back applies the saved preference without requiring the Activity to be created again.

Verification results

The debug APK compiled successfully. All 14 JVM matching tests passed with zero failures or errors. All five Android instrumented database tests passed on API 35. Those tests cover CRUD after closing and reopening the helper, exactly 20 seeded recipes, missing record handling, sort order, invalid quantities and suggestions changing with database updates.

25 additional checks passed against the running interface and its persisted database. The checks included blank-name and zero-quantity errors, missing and insufficient ingredients, unit conversion, create, read, edit, confirmed deletion, cancelled deletion, settings through Back, force-stop persistence, rotation, larger font sizing and foreign-key integrity. The test record is included in docs/VERIFICATION.md.

5 Challenges and solutions

Keeping suggestions strict

Matching only ingredient names would incorrectly suggest Banana milk when only 200 ml of milk is available. Comparing raw unit strings would also reject 0.25 l against a requirement of 250 ml. The solution was to compare compatible units in a shared base unit and reject the entire recipe on the first shortage. Both database integration tests and the actual interface sequence verified the boundary between 0.2 l and 0.25 l.

Refreshing records without losing form edits

A screen can return from the back stack without a new onCreate call. Loading a pantry list only in onCreate would leave stale quantities or sort order. The list and recipe screens therefore reload in onResume. The form uses its saved state guard when loading an existing ingredient, preventing a rotation from overwriting an unsaved change with the old database value.

Keeping the app within the course scope

The stored recipe collection had to be readable without adding a cloud service or unfamiliar persistence framework. It is seeded with Java arrays and SQLite inserts. Shared behaviour is kept in one small base Activity, while the screen-specific code follows the same widget, listener and Intent patterns as the course contact examples. The interface uses system-bar padding so the modern target SDK does not place controls underneath navigation bars.

6 Conclusion and reflection

The implemented system covers the required five screens, persistent pantry CRUD, custom list adapters, strict suggestions, recipe details and saved settings. The main lesson is that a displayed list must reflect the actual stored quantities and current lifecycle state, rather than only the values present when a screen first opens.

The implementation deliberately uses a limited alias list and three measurement groups. It does not convert arbitrary ingredient mass into volume or recognise every possible ingredient phrase. A future extension could broaden the curated aliases or add optional expiry dates after the required app has been assessed. A future database version would need an explicit migration; the current helper refuses an unsupported upgrade rather than deleting existing pantry records.

Before submission, the student must complete the institutional cover fields and signature, review and acknowledge the assistance used, grant marker access to the private repository and record the narrated video. The app and report do not replace that required personal demonstration.

7 References

Google (n.d.a) Save data using SQLite. Android Developers. Available at: https://developer.android.com/training/data-storage/sqlite (Accessed: 25 October 2026).

Google (n.d.b) Create dynamic lists with RecyclerView. Android Developers. Available at: https://developer.android.com/develop/ui/views/layout/recyclerview (Accessed: 25 October 2026).

Google (2026) Android Gradle plugin 9.4.0. Android Developers. Available at: https://developer.android.com/build/releases/agp-9-4-0-release-notes (Accessed: 25 October 2026).

OpenAI (2026) Codex. AI assistance used for source implementation, test execution and report preparation in this project. Available at: https://openai.com/codex/ (Used: 25 October 2026).

Richfield Graduate Institute of Technology (n.d.a) Mobile App Development 700 Practical Assignment Smart Pantry Manager. Supplied assignment PDF.

Richfield Graduate Institute of Technology (n.d.b) Mobile App Development 700 Learner Guide. Supplied module PDF. Chapters 1 to 5.

ContactList App Multi View Development Manual (n.d.) Supplied contact-list-multiview-guide.pdf. Supporting Java and XML examples. Authorship and publication date were not established from the supplied file.